

# Plano — agregação por portadora (EARFCN) a nível de site, com 5G

---

Documento de handoff. Escrito para ser lido por uma sessão nova, sem contexto anterior. Data: 2026-08-28. Origem do pedido: `portadora.md`.

## 1. Contexto

---

### O que já existe (não refazer)

Implementado em 2026-08-26 (ver `MEMORY.md`, seção "Clusters por portadora (DLEARFCN)"):

- `server.py::parse_sites` aceita as colunas `dlearfcn` / `earfcn` / `dl_earfcn` da EP e grava `cell.earfcn` como inteiro em texto (`server.py::_normalize_earfcn`, linha ~55).
- `Api._earfcn_clusters` (`api/api.py:777`) gera **clusters sintéticos globais**, um por valor de DLEARFCN, com recorte granular de células (`members[].cell_ids`). Id estável `earfcn-<valor>`, nome `Portadora <valor>`, `source: "earfcn"`.
- `Api._clusters_of` (`api/api.py:821`) devolve clusters cadastrados + os gerados. Um cluster salvo com o mesmo id ganha da geração (o operador pode ter editado nome/cor).
- A Visão Geral de KPIs 4G abre com essas portadoras marcadas (`frontend/js/kpi_overview.js::_applyPreferredSelection`, linha 206).
- Hoje 5G é explicitamente excluído: `api/api.py:792` (`if cls._cell_technology_family(cell) == "5G": continue`).

### O que o chefe pediu agora

1. "Tem que ter essa agregação a nível de EARFCN **por site** também — quando filtrar o site, conseguir ver o balanceamento dele por EARFCN."
2. O 5G entra no jogo: o NR-ARFCN será preenchido **na mesma coluna** `DLEARFCN` da EP.

### Achado relevante

`server.py::_normalize_earfcn` já aceita NR-ARFCN. O código não tem teto de valor — só o docstring fala em "0–65535". Nenhuma mudança de parsing é necessária; só o docstring e um teste.

## 2. Abordagem escolhida

---

B + C, decidido junto com o usuário:

- **B (mecanismo):** novo tipo de escopo `site_carrier` na API, com `scope_id` composto `<site_id>::<earfcn>`. Resolve para uma seleção parcial de site (as células daquela portadora) e reaproveita `_cluster_series_by_family` — **nenhuma agregação nova é escrita**.
- **C (UX):** no seletor de Sites da Visão Geral, cada site com  $\geq 2$  portadoras ganha expansão “separar por portadora”; cada portadora vira uma série comparável.

**Rejeitado:** gerar clusters sintéticos `site×portadora` ( `earfcn-1276@725483` ) — explosão combinatória no dropdown do dashboard, afogaria os clusters geográficos que o operador monta à mão, que é justamente a fronteira preservada em `_clusters_of`.

## Decisões travadas com o usuário

1. **Namespace de id único:** 5G usa `earfcn-<valor>`, igual ao 4G. Partimos do pressuposto de que os espaços numéricos não colidem (LTE EARFCN  $\leq \sim 65k$ ; NR-ARFCN de FR1 realista  $\geq \sim 140k$ ). Sem prefixo `nrarfcn-`.
2. **A aba 5G abre com as portadoras marcadas** por padrão, igual à 4G.
3. **Site e portadoras dele podem coexistir** na comparação (total + partes) — é o que torna o balanceamento legível.
4. **Dashboard/mapa não mudam** nesta entrega. O escopo `site_carrier` fica disponível em `get_kpi_series`, então o popup de gráfico do dashboard pode adotá-lo depois sem backend novo.

## 3. Fase 0 — Abrir o 5G nas portadoras globais

Sem isso não há o que agrupar por site no 5G.

### 0.1 `api/api.py::_earfcn_clusters` (linha 777)

- Remover o `continue` do 5G (linha 792).
- Resolver a família de cada célula como `cls._cell_technology_family(cell)` or `self._single_configured_family(config)`. O fallback importa: há eventos cujas células não carregam `4G / 5G` no nome (ver comentário sobre Curitiba em `_configured_kpi_technologies`, `api/api.py:1131`).
- Continuar agrupando **por valor de earfcn** (decisão 1), mas derivar a família do grupo: `familias = {família resolvida de cada célula} - {None}`; `family =` único elemento se `len == 1`, senão `None`.
- Cada cluster gerado ganha o campo `"family"` ( `"4G"`, `"5G"` ou `None` ).
- **Cor:** hoje o índice de `_EARFCN_CLUSTER_COLORS` é global sobre a lista ordenada. Passar a enumerar **por família** (4G conta do zero, 5G conta do zero). Sem isso, portadoras 4G existentes podem repintar quando o 5G entrar na lista — o usuário perceberia como “os gráficos trocaram de cor”.

### 0.2 `api/api.py::get_clusters` (linha 1087)

Propagar `family` no `entry`, ao lado de `source`, quando presente.

## 0.3 Verificação

Evento com célula 4G `earfcn=1276` e célula 5G `earfcn=627264` gera dois clusters; cada um só com as suas células; `get_kpi_series(scope="cluster", scope_id="earfcn-627264", technology_family="5G")` retorna só linhas 5G.

## 0.4 Atenção — reversão de decisão testada

`tests/test_api.py:846` ( `test_ao_mistura_celulas_5g_mesmo_com_earfcn_preenchido` ) afirma exatamente o oposto ( `assert "earfcn-627264" not in ids` ). Reescrever como

`test_5g_gera_portadora_propria_sem_misturar_com_4g` : os dois clusters existem, cada um com as células da sua família. **Não apagar em silêncio** — a reversão vai documentada na Fase 4.

---

## 4. Fase 1 — Backend: escopo `site_carrier`

---

Contrato: `scope="site_carrier", scope_id="<site_id>::<earfcn>"`.

### 1.1 Novo helper `Api._site_carrier_selections(config, merged, site_id, earfcn, family)`

Colocar ao lado de `_cluster_raw_selections` ( `api/api.py:671` ). Deve:

1. achar o site fundido com `_find_merged_site` (já trata id legado de gêmeo 4G/5G);
2. filtrar as células por `earfcn` e pela família resolvida (mesma regra da Fase 0.1);
3. mapear cada célula ao `site_id` bruto dono via `_owner_site_id_for_cell` ( `api/api.py:1203` ) — obrigatório: `_cluster_series_by_family` espera ids de membro, não o id fundido;
4. devolver `[{"site_id": <bruto>, "cell_ids": [...]}]`.

### 1.2 `api/api.py::get_kpi_series` (linha 1413)

Novo ramo `if scope == "site_carrier":`, espelhando o ramo `scope == "cluster"` (linha 1444): monta `selections` pelo helper novo e chama `_cluster_series_by_family`. Escopo sem células casadas devolve a série vazia padrão ( `labels: [], values: [], series: [] + thresholds` ), como o cluster inexistente já faz.

### 1.3 `get_kpi_overview` (1564) e `get_kpi_overview_multi` (1662)

Incluir `"site_carrier"` nas validações de escopo (as ocorrências de `in ("site", "cluster", "cell")`) e no repasse de `scope / scope_id` para `get_kpi_series` — o `site_carrier` segue o mesmo caminho de `cluster / cell` (passa por `scope`, não por `site_id`).

### 1.4 `api/api.py::get_sites` (dict de saída na linha ~975)

Novo campo por site:

```
"carriers": [{"earfcn": "1276", "family": "4G", "cell_count": 3}, ...]
```

Derivado de `visible_cells` (respeita o mesmo filtro de família da chamada), ordenado numericamente. Alimenta o seletor do frontend; evita endpoint novo e evita reimplementar a classificação de família em JS. Incluir também no `fallback` do `except` para a lista não perder o campo em degradação.

## 1.5 Verificação

- `get_kpi_overview(scope="site_carrier", scope_id="<site>::1276")` bate exatamente com o agregado calculado a partir de `scope="cell"` das células daquela portadora (mesma grade, mesmos valores).
- Em site gêmeo 4G/5G, `::<nrarfcn>` só puxa o membro 5G.

## 5. Fase 2 — Frontend

### 2.1 `frontend/js/kpi_overview.js`

- `_selection` (linha 61) ganha `siteCarrier: new Set()`, chave `<site_id>::<earfcn>`. Entra em `_selectionSize()`, `_selectionKey()` e nas purgas de `_refreshScopeData` (linha 666).
- `_selectedScopes()` (linha 278): novo bloco depois de sites, antes de células. `scope: "site_carrier"`, `name: "<site> · <earfcn>"`. Cor-base = a cor do cluster de portadora correspondente (`earfcn-<v>`), passando pelo mesmo mecanismo `taken` que já resolve colisão. Assim `Portadora 1276` e `SPPNB2 · 1276` saem na mesma cor quando ela está livre, e nunca colidem quando não está.
- `_carrierClusters()` (linha 201): filtrar por família — `!cluster.family || cluster.family === _family`. Portadora de família indeterminada continua visível nas duas abas (comportamento de hoje).
- `_applyPreferredSelection()` (linha 206): trocar `_family === "4G" && carriers.length` por só `carriers.length` (decisão 2).
- `_renderClusterPicker()`: iterar sobre a lista filtrada por família em vez de `_scopeClusters` cru — senão "Todos os clusters" marca portadoras da outra aba, que renderizariam vazias.
- `_renderSiteOptions()` (linha 445) — o coração do C: site com  $\geq 2$  portadoras na família atual ganha um chevron "separar por portadora". Expandido, mostra uma sub-linha por portadora (checkbox + swatch + `N células`). O checkbox do site continua independente (decisão 3).
- `_cellScopeSiteIds()` (linha 508): incluir os sites com portadora marcada, para o seletor de células continuar coerente.
- `_syncPickerSummaries()` (linha 585): resumo do seletor de sites conta sites + portadoras (ex.: `"2 sites · 3 portadoras"`).
- Teto de escopos (`MAX_SCOPES = 8`): o `blocked` existente cobre as sub-linhas. Ao expandir um site pela primeira vez, marcar todas as portadoras dele **limpando as seleções anteriores** se não couberem — expandir tem que simplesmente funcionar, não bater no limite em silêncio.

2.2 frontend/js/bridge.js

- carriers nos MOCK\_SITES .
- MOCK\_EARFCN\_CLUSTERS com family e uma portadora 5G ( earfcn-627264 , family: "5G" ).
- scopeOffset do mock de get\_kpi\_overview reconhecendo scope === "site\_carrier" , para as séries saírem visivelmente distintas no modo mock.

2.3 frontend/css/main.css

Estilo das sub-linhas (indentação + chevron), reaproveitando .scope-picker-option .

6. Fase 3 — Testes

| Arquivo                                        | Cobre                                                                                                                                         |
|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| tests/test_server_parse_sites.py               | NR-ARFCN de 6 dígitos ( 627264 ) é aceito e gravado — fecha o buraco entre docstring e código                                                 |
| tests/test_api.py::TestEarfcnClusters          | reescrever o teste do 5G (\$0.4); family no get_clusters ; cores 4G não repintam ao entrar 5G                                                 |
| tests/test_api.py::TestSiteCarrierScope (novo) | resolução do escopo; site gêmeo; earfcn inexistente → série vazia; get_kpi_overview_multi aceitando o escopo; carriers no get_sites           |
| tests/test_frontend_kpi_overview_ui.py         | novo cenário ?kpi0verview=earfcn5g ; expandir site → chips SPPNB2 · 1276 e · 1700 com cores distintas; aba 5G abrindo com portadoras marcadas |

Gate (CLAUDE.md §4): pytest tests/ -q --ignore=tests/test\_http\_vpn.py . Baseline conhecida: 546 passed. Meta: 546 + novos, zero regressões. Depois, python main.py --mock --dev para conferir a expansão na tela.

7. Fase 4 — Documentação

Entrada nova em MEMORY.md com:

- a reversão do “5G fica de fora” e o porquê (o cliente vai popular NR-ARFCN na mesma coluna DLEARFCN );
- a decisão de namespace de id único earfcn-<valor> para as duas famílias, e o pressuposto de não-colisão que a sustenta;
- o contrato do escopo site\_carrier ( <site\_id>::<earfcn> ) e o fato de ele reusar \_cluster\_series\_by\_family sem agregação nova;
- a enumeração de cor por família.

`ERRORS.md` só se algo quebrar no caminho.

---

## 8. Armadilhas conhecidas

---

- `_cluster_series_by_family` espera **ids de site brutos** (membros), não o id fundido. Passar o id fundido faz a série voltar vazia sem erro.
- `_cell_technology_family` devolve `None` quando o nome da célula não carrega `4G` / `5G`. Não tratar `None` como 4G cegamente — usar o fallback `_single_configured_family`, senão um evento 5G-only sem token no nome perde as portadoras na aba 5G.
- A cor do cluster gerado indexada globalmente repinta as portadoras 4G quando o 5G entra. Enumerar por família (§0.1).
- Separador `::` no `scope_id`: ids de site do evento não contêm `::` hoje. Ainda assim, fazer `rsplit("::", 1)` para o earfcn ser sempre o último segmento.
- O `_pendingSeed` de `_refreshScopeData` reaplica a semente quando a seleção fica vazia; ao limpar portadoras, garantir que isso não reintroduza escopos que o usuário acabou de desmarcar (foi exatamente o motivo de `_refreshCellsForFamily` existir separado).

## 9. Checklist de execução

---

- ☐ [] Fase 0 — 5G nas portadoras globais + `family` no `get_clusters` + cor por família
- ☐ [] Fase 0.4 — reescrever `test_ao_mistura_celulas_5g_mesmo_com_earfcn_preenchido`
- ☐ [] Fase 1.1 — `_site_carrier_selections`
- ☐ [] Fase 1.2 — ramo `site_carrier` em `get_kpi_series`
- ☐ [] Fase 1.3 — `site_carrier` em `get_kpi_overview` e `get_kpi_overview_multi`
- ☐ [] Fase 1.4 — `carriers` no `get_sites` (+ fallback)
- ☐ [] Fase 2.1 — seleção, cores e sub-linhas em `kpi_overview.js`
- ☐ [] Fase 2.2 — mocks em `bridge.js`
- ☐ [] Fase 2.3 — CSS das sub-linhas
- ☐ [] Fase 3 — testes (Python + Playwright)
- ☐ [] Gate — `pytest tests/ -q --ignore=tests/test_http_vpn.py` verde
- ☐ [] Smoke — `python main.py --mock --dev`
- ☐ [] Fase 4 — `MEMORY.md`